ĐẠI HỌC QUỐC GIA TP HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN



MÔN KỸ THUẬT XÂY DỰNG HỆ THỐNG AGENTIC AI

BÀI TẬP VỀ NHÀ 3

XÂY DỰNG AGENT ĐẶT VÉ MÁY BAY BẰNG LANGCHAIN VÀ LANGGRAPH

LỚP SE373.R11

NGÀY NỘP: Chưa xác định trong tài liệu đề bài

HỌ VÀ TÊN SINH VIÊN: HOÀNG KHÔI NGUYÊN

MSSV: 24521176

1. Giới thiệu

Project xây dựng Agent hỗ trợ đặt vé nội địa một chiều trên môi trường mock. Người dùng cung cấp route, ngày bay, số khách, cabin, ngân sách tổng và chính sách chọn chuyến. Hệ thống chỉ tạo booking khi đã có xác nhận phù hợp và kiểm tra kết quả bằng code.

Bài tập tập trung vào Harness và ba mẫu ReAct, Plan-then-Execute, Hybrid. Phiên bản hiện tại chạy được offline, có 82 test pass và benchmark 90 lượt. Số liệu dùng để kiểm chứng orchestration và cơ chế an toàn; chưa đánh giá reasoning của LLM thật.

2. Công nghệ sử dụng

Công nghệ	**Vai trò**
Python 3.12	Môi trường đã chạy và kiểm thử
LangChain 1.4.3	create_agent, middleware, BaseChatModel, tool calling
LangGraph 1.2.13	StateGraph và conditional edges cho executor/re-plan
Pydantic 2.13.5	Schema request, approval và structured plan
pytest 9.1.1	82 unit/integration tests
python-dotenv; langchain-openai	Đọc .env và adapter LLM tùy chọn, chưa chạy live
Scripted model	Policy deterministic để test offline, không phải LLM

3. Chức năng chính

- Tiếp nhận yêu cầu và giữ constraints dưới dạng dữ liệu frozen.
- Tìm chuyến, đọc chi tiết, kiểm ghế và tính tổng tiền theo số khách.
- Yêu cầu xác nhận cho từng quote trước khi ghi booking.
- Đọc lại booking DB để kiểm completion; handoff khi cần người can thiệp.
- Chạy ba architecture trên cùng dữ liệu và ghi trace/metrics để so sánh.

4. Thiết kế hệ thống

4.1. Dữ liệu và Mock Tools

MockFlightDatabase chứa 10 chuyến tại SGN, HAN, DAD trong các ngày 15–16/10/2026. Booking lưu in-memory; mỗi test và mỗi lượt evaluation tạo DB riêng. Tiền dùng số nguyên VND; ngân sách là tổng booking, không phải giá mỗi khách.

Tool	Chức năng
search_flights	Tìm đúng route/ngày/cabin/hãng/ngân sách tổng
get_flight_details	Lấy thông tin chuyến theo flight_id
check_seat_availability	Kiểm số ghế đáp ứng số khách
calculate_price	Tính base_price × passenger_count
book_flight	Ghi booking CONFIRMED và trừ ghế sau permission

Luồng chính: User → Agent → BookingHarness → Mock Tools → Mock Database. Bốn tool đầu đọc dữ liệu; book_flight tạo side effect. Agent không được gọi raw booking adapter để bỏ qua Harness.

4.2. Harness

Ràng buộc là dữ liệu. BookingRequest frozen lưu origin, destination, departure_date, passenger_count, cabin_class, airline, max_budget và selection_policy. Harness kiểm route, lịch, số khách, ngân sách, schema tool arguments và đối chiếu mọi action với request; không tự nới ngân sách hoặc đổi ngày.

Kiểm quyền. READ tools chạy sau validation. Approval lưu fingerprint toàn bộ request, flight_id, số khách và tổng tiền. Approval chuyến A không dùng cho B; giá hoặc số khách thay đổi làm approval không còn phù hợp. Callback xác nhận thuộc application, không phải model.

Completion bằng code. Hệ thống đọc booking DB và audit approval; kiểm request, chuyến, route/ngày, cabin/hãng, hành khách, giá, ngân sách và status. Model tự nói đã đặt vé không làm tác vụ hoàn thành.

Handoff. Gói bàn giao gồm stop_reason, da_thu, trang_thai, cau_hoi_cho_nguoi. Giới hạn tài nguyên: 40 tool proposals, tối đa 2 replans và graph recursion limit. Validator dùng ngày tham chiếu cố định 06/10/2026 để tái lập fixture bài học.

5. Ba mẫu thiết kế Agent

5.1. ReAct

create_agent điều phối vòng action → observation → action tiếp theo. ScriptedBookingModel kế thừa BaseChatModel, chọn tool dựa trên observation trước đó. Middleware chuyển mọi tool proposal sang BookingHarness.execute. Khi chuyến hết ghế, policy thử lại với candidate khác và yêu cầu approval mới.

5.2. Plan-then-Execute

Planner trả structured Plan gồm tám bước: search, select candidate, details, seats, price, confirmation, booking, verification. Executor StateGraph thực thi từng bước. Plan chỉ cho registered tools và ba control actions; không sinh hay execute Python. Baseline lập plan một lần và handoff khi plan mất hiệu lực.

5.3. Hybrid

Hybrid dùng planner như PTE nhưng kiểm observation sau từng bước. Khi candidate hết ghế, giá thay đổi hoặc tool lỗi có thể phục hồi, conditional edge quay lại planner. Candidate lỗi bị loại, approval cũ bị hủy, tối đa hai lần re-plan. Thay constraint cần request/session mới; chưa có UI sửa constraint giữa phiên.

Tiêu chí	ReAct	PTE	Hybrid
Plan ban đầu	Không	Có	Có
Sau observation	Chọn action mới	Theo plan	Kiểm rồi đi tiếp
Khi S08 hết ghế	Thử alternative	Plan mất hiệu lực	Re-plan
Planner calls	Không riêng	Một lần	Có thêm khi lỗi

6. Quy trình hoạt động

1. Nhập request và tạo BookingRequest.
2. Harness kiểm input và giữ constraints frozen.
3. Agent lập plan hoặc đề xuất action theo architecture.
4. Harness kiểm tool name, schema và constraints.
5. Tool thực thi trên DB, trả observation và ghi trace.
6. Chọn candidate theo policy hoặc hỏi người dùng.
7. Kiểm ghế, tính tổng tiền, xin xác nhận đúng quote.
8. Harness kiểm approval trước book_flight.
9. Tạo booking và trừ ghế; nếu môi trường đổi thì xử lý theo architecture.
10. CompletionChecker đọc DB/audit; trả COMPLETED hoặc handoff.

7. Kết quả chạy demo

7.1. ReAct Agent

S01 yêu cầu SGN → HAN ngày 15/10/2026, Economy, một khách, ngân sách 2.000.000 VND và chọn rẻ nhất. Agent chọn VJ120 giá 1.200.000 VND, lấy xác nhận, tạo BK001 và verification pass. Ảnh dưới là phần trace được render từ JSON của lần chạy thật; approval trong smoke run là người dùng giả lập.

ReAct / S01 - actual captured trace

```
REQUEST S01: SGN -> HAN 2026-10-15
Cabin=ECONOMY passengers=1 budget=2,000,000 VND
01. search_flights {"origin": "SGN", "destination": "HAN", "departure_date": "2026-10-15",
"cabin_class": "ECONOMY", "airline": null, "max_budget": 2000000}
    ALLOW -> {"status": "ok", "candidates": ["VN210", "VJ120", "QH202"]}
02. get_flight_details {"flight_id": "VJ120"}
    ALLOW -> {"status": "ok"}
03. check_seat_availability {"flight_id": "VJ120", "passenger_count": 1}
    ALLOW -> {"status": "ok", "available": true}
04. calculate_price {"flight_id": "VJ120", "passenger_count": 1}
    ALLOW -> {"status": "ok", "total_price": 1200000}
05. USER confirm VJ120 total=1,200,000: True
06. book_flight {"flight_id": "VJ120", "passenger_count": 1, "passenger_name": "Hoang Khoi
Nguyen", "total_price": 1200000}
    ALLOW -> {"status": "CONFIRMED", "booking_id": "BK001"}
07. verify_completion {"booking_id": "BK001"}
    PASS -> {"completed": true}
FINAL: COMPLETED | booking_id=BK001
DB bookings=1 | tool calls=5 | model calls=6
```

Hình 1. Trace ReAct S01 với booking và completion được kiểm bằng code.

Trace ghi action, arguments, Harness decision và observation. Không ghi chain-of-thought. CLI tương tác dùng cùng flow nhưng chờ người dùng nhập y/n ở bước xác nhận.

7.2. Plan-then-Execute Agent

S01 cho kết quả giống ReAct nhưng cách điều phối khác: planner tạo tám bước trước khi executor chạy. Model không chọn lại action ở mỗi tool observation. Kế hoạch được kiểm theo allowlist và thứ tự an toàn trước khi thực thi.

Plan-then-Execute / S01 - actual captured trace

```
REQUEST S01: SGN -> HAN 2026-10-15
Cabin=ECONOMY passengers=1 budget=2,000,000 VND
01. planner -> search_flights > select_candidate > get_flight_details >
check_seat_availability > calculate_price > request_confirmation > book_flight >
verify_completion
02. search_flights {"origin": "SGN", "destination": "HAN", "departure_date": "2026-10-15",
"cabin_class": "ECONOMY", "airline": null, "max_budget": 2000000}
    ALLOW -> {"status": "ok", "candidates": ["VN210", "VJ120", "QH202"]}
03. get_flight_details {"flight_id": "VJ120"}
    ALLOW -> {"status": "ok"}
04. check_seat_availability {"flight_id": "VJ120", "passenger_count": 1}
    ALLOW -> {"status": "ok", "available": true}
05. calculate_price {"flight_id": "VJ120", "passenger_count": 1}
    ALLOW -> {"status": "ok", "total_price": 1200000}
06. USER confirm VJ120 total=1,200,000: True
07. book_flight {"flight_id": "VJ120", "passenger_count": 1, "passenger_name": "Hoang Khoi
Nguyen", "total_price": 1200000}
    ALLOW -> {"status": "CONFIRMED", "booking_id": "BK001"}
08. verify_completion {"booking_id": "BK001"}
    PASS -> {"completed": true}
FINAL: COMPLETED | booking_id=BK001
DB bookings=1 | tool calls=5 | model calls=1
```

Hình 2. Structured plan và trace thực thi PTE S01.

Ở S08, cùng plan này không còn thực hiện được vì chuyến đã hết ghế trước booking. Baseline dừng PLAN_INVALIDATED, DB không có booking. Đây là handoff an toàn nhưng chưa đạt mục tiêu booking của scenario.

7.3. Hybrid Agent

S08 mô phỏng ghế của VJ120 chuyển về 0 ngay trước ghi booking. Tool trả SEATS_NOT_AVAILABLE. Hybrid phát hiện lỗi, hủy approval cũ, loại VJ120 và gọi planner lần thứ hai. Candidate thay thế QH202 có tổng tiền 1.500.000 VND nên phải được xác nhận lại trước khi đặt.

Hybrid / S08 - actual captured trace

```
REQUEST S08: SGN -> HAN 2026-10-15
Cabin=ECONOMY passengers=1 budget=2,000,000 VND
01. planner -> search_flights > select_candidate > get_flight_details >
check_seat_availability > calculate_price > request_confirmation > book_flight >
verify_completion
02. search_flights {"origin": "SGN", "destination": "HAN", "departure_date": "2026-10-15",
"cabin_class": "ECONOMY", "airline": null, "max_budget": 2000000}
    ALLOW -> {"status": "ok", "candidates": ["VN210", "VJ120", "QH202"]}
03. get_flight_details {"flight_id": "VJ120"}
    ALLOW -> {"status": "ok"}
04. check_seat_availability {"flight_id": "VJ120", "passenger_count": 1}
    ALLOW -> {"status": "ok", "available": true}
05. calculate_price {"flight_id": "VJ120", "passenger_count": 1}
    ALLOW -> {"status": "ok", "total_price": 1200000}
06. USER confirm VJ120 total=1,200,000: True
07. book_flight {"flight_id": "VJ120", "passenger_count": 1, "passenger_name": "Hoang Khoi
Nguyen", "total_price": 1200000}
    ALLOW -> {"status": "error", "error_code": "SEATS_NOT_AVAILABLE"}
08. planner -> search_flights > select_candidate > get_flight_details >
check_seat_availability > calculate_price > request_confirmation > book_flight >
verify_completion
09. search_flights {"origin": "SGN", "destination": "HAN", "departure_date": "2026-10-15",
"cabin_class": "ECONOMY", "airline": null, "max_budget": 2000000}
    ALLOW -> {"status": "ok", "candidates": ["VN210", "VJ120", "QH202"]}
10. get_flight_details {"flight_id": "QH202"}
    ALLOW -> {"status": "ok"}
11. check_seat_availability {"flight_id": "QH202", "passenger_count": 1}
    ALLOW -> {"status": "ok", "available": true}
12. calculate_price {"flight_id": "QH202", "passenger_count": 1}
    ALLOW -> {"status": "ok", "total_price": 1500000}
13. USER confirm QH202 total=1,500,000: True
14. book_flight {"flight_id": "QH202", "passenger_count": 1, "passenger_name": "Hoang Khoi
Nguyen", "total_price": 1500000}
    ALLOW -> {"status": "CONFIRMED", "booking_id": "BK001"}
15. verify_completion {"booking_id": "BK001"}
    PASS -> {"completed": true}
FINAL: COMPLETED | booking_id=BK001
DB bookings=1 | tool calls=10 | model calls=2
```

Hình 3. Hybrid re-plan và lấy approval mới cho QH202 trong S08.

BK001 cuối cùng thuộc QH202, verification pass. Test đối chiếu hai quote được confirm lần lượt là VJ120 và QH202; không tái sử dụng approval chuyến cũ.

7.4. Harness chặn hành động không hợp lệ

Thử trực tiếp gọi book_flight khi chưa có approval. Harness trả BLOCK và USER_CONFIRMATION_REQUIRED, DB vẫn rỗng. Sau đó giả sử model nói đã đặt BK001; verification đọc DB trả False. Bằng chứng được lưu trong results/safety_demo.json.

Direct Harness negative test

```
Attempt: {"flight_id": "VJ120", "passenger_count": 1, "passenger_name": "Hoang Khoi Nguyen",
"total_price": 1200000}
Harness decision: BLOCK
Observation: {"status": "blocked", "error_code": "USER_CONFIRMATION_REQUIRED"}
Booking DB: []
Claimed booking: BK001 (model text only)
Completion by code: False
```

Hình 4. Harness chặn booking chưa approval và từ chối false completion.

S05 thực hiện các bước tìm kiếm, kiểm ghế và quote, nhưng người dùng chưa xác nhận. Agent dừng NEED_USER_CONFIRMATION và cung cấp gói handoff đủ bốn trường.

ReAct / S05 - approval pending

```
Request: {"request_id": "S05", "origin": "SGN", "destination": "HAN", "departure_date":
"2026-10-15", "passenger_count": 1, "passenger_name": "Hoang Khoi Nguyen", "max_budget":
2000000, "cabin_class": "ECONOMY", "airline": null, "selection_policy": "cheapest",
"selected_flight": null}
Status: NEED_USER_CONFIRMATION
stop_reason: NEED_USER_CONFIRMATION
trang_thai: {"selected_flight": "VJ120", "quote": {"flight_id": "VJ120", "passenger_count": 1,
"total_price": 1200000, "request_id": "S05"}, "booking_id": null, "bookings": []}
cau_hoi_cho_nguoi: Xác nhận đặt vé với đúng chuyến, số khách và tổng tiền trên?
DB bookings: 0
```

Hình 5. Handoff S05 khi approval pending và booking DB rỗng.

Các test bổ sung kiểm approval sai chuyến, request khác, giá/số khách thay đổi, constraints bị đổi, booking bị sửa và repeated delivery. Cùng request/quote được giao lại trong một session không tạo thêm booking; chưa có resume lưu bền giữa process.

8. Đánh giá ba mẫu Agent

Thiết kế: 10 scenario S01–S10, ba architecture, ba lần lặp; tổng 90 lượt offline. Mỗi lượt reset DB độc lập. S01/S08/S09 có mục tiêu booking; các scenario còn lại cần handoff an toàn. Test: 82 passed, 0 failed. Kết quả dưới lấy trực tiếp từ evaluate.py và results/evaluation.json.

Scenario	Mục tiêu
S01	Cheapest happy path
S02	Hỏi chọn khi thiếu policy
S03	Thiếu ngày: không mutation
S04	Không có chuyến: không đổi ngày
S05/S06	Pending / từ chối approval
S07	Chuyến chỉ định hết ghế
S08	Ghế mất ngay trước booking
S09/S10	Hai khách: đủ / thiếu ngân sách tổng

Metric	ReAct	PTE	Hybrid
Task Success Rate	100.00%	90.00%	100.00%
Booking Success Rate	100.00%	66.67%	100.00%
Constraint Violation Rate	0.00%	0.00%	0.00%
Permission Violation Rate	0.00%	0.00%	0.00%
False Completion Rate	0.00%	0.00%	0.00%
Tool calls trung bình	3.50	2.90	3.50
Model calls trung bình	4.40	0.90	1.10
Execution time TB (ms)	34.37	26.39	26.87
Handoff đúng / tổng	21/21	24/24	21/21
Token usage	N/A	N/A	N/A

Task Success Rate tính đúng outcome / 30 lượt mỗi architecture, gồm safe handoff. Booking Success Rate dùng 9 lượt S01/S08/S09. PTE S08 được tính handoff đúng nhưng không tính task success. Tool calls gồm proposals bị chặn. Model calls offline là scripted decisions hoặc planner calls; không phải token/chi phí LLM.

Measured benchmark / 10 scenarios x 3 repeats

```
architecture     task%   booking%  tools  model  elapsed_ms  violations
react            100.0    100.00   3.50   4.40      34.37  0 / 0 / 0
plan_execute      90.0     66.67   2.90   0.90      26.39  0 / 0 / 0
hybrid           100.0    100.00   3.50   1.10      26.87  0 / 0 / 0
Booking denominator: S01, S08, S09 (9 runs per architecture).
Violations: constraint / permission / false completion.
Mode: offline scripted model. Tokens: N/A. No provider latency measured.
```

Hình 6. Bảng benchmark được render từ output evaluation thực tế.

9. So sánh và nhận xét

ReAct và Hybrid đạt task success 100% trong bộ scenario; PTE đạt 90%. Khác biệt đến từ S08: PTE không re-plan nên dừng an toàn, còn ReAct và Hybrid đặt được alternative. Booking success tương ứng là 100%, 66,67% và 100% trên các scenario có mục tiêu booking.

PTE có ít tool/model calls hơn vì không tiếp tục sau plan invalidation. Hybrid chỉ gọi planner khi cần; ReAct dùng scripted model để quyết định action sau mỗi observation. Cả ba không có vi phạm constraint/permission hay false completion trong 90 lượt đã đo. Kết quả không chứng minh khả năng reasoning của một architecture luôn tốt hơn.

Thời gian đo gồm dựng graph và chạy mock tool, không gồm nhập CLI hoặc gọi provider thật. Ba lần lặp kiểm tái lập của orchestration; không đại diện độ đa dạng đầu ra của LLM hoặc phân phối độ trễ API.